

Kafka

[Apache Kafka](#) is an open-source distributed event streaming platform that can be used either as a [message queue](#) or as a [stream processing system](#). Kafka excels in delivering high performance, scalability, and durability. It's engineered to handle vast volumes of data in real-time, ensuring that no message is ever lost and that each piece of data is processed as swiftly as possible.

In this deep dive, we're going to take a top down approach. Starting with a zoomed out view of Kafka and progressing into more and more detail.

A Motivating Example

It's the World Cup (my personal favorite competition). And we run a website that provides real-time statistics on the matches. Each time a goal is scored, a player is booked, or a substitution is made, we want to update our website with the latest information.

Events are placed on a queue when they occur. We call the server or process responsible for putting these events on the queue the **producer**. Downstream, we have a server that reads events off the queue and updates the website. We call this the **consumer**.

Now, imagine the World Cup expanded from just the top 48 teams to a hypothetical 1,000-team tournament, and all the games are now played at the same time. The number of events has increased significantly, and our single server hosting the queue is struggling to keep up. Similarly, our consumer feels like it has its mouth under a firehose and is crashing under the load.

We need to scale the system by adding more servers to distribute our queue. But how do we ensure that the events are still processed in order?

If we were to randomly distribute the events across the servers, we would have a mess on our hands. Goals would be scored before the match even started, and players would be booked for fouls they haven't committed yet.

A logical solution is to distribute the items in the queue based on the game they are associated with. This way, all events for a single game are processed in order because they exist on the same queue. This is one of the fundamental ideas behind Kafka: **messages sent and received through Kafka require a user specified distribution strategy.**

But what about our consumer, it's still overwhelmed. It is easy enough to add more, **but how do we make sure that each event is only processed once?** We can group consumers together into what Kafka calls a **consumer group**. With consumer groups, each event is guaranteed to only be processed by one consumer in the group.

Lastly, we've decided that we want to expand our hypothetical World Cup to more sports, like basketball. But we don't want our soccer website to cover basketball events, and we don't want our basketball website to cover soccer events. So we introduce the concept of **topics**. **Each event is associated with a topic, and consumers can subscribe to specific topics.** Therefore, our consumers who update the soccer website only subscribe to the soccer topic, and our consumers that update the basketball website only subscribe to basketball events.

Basic Terminology and Architecture

The example is great, but let's define Kafka a bit more concretely by formalizing some of the key terms and concepts introduced above.

A Kafka cluster is made up of multiple **brokers**. These are just individual servers (they can be physical or virtual). Each broker is responsible for storing data and serving clients. The more brokers you have, the more data you can store and the more clients you can serve.

Each broker has a number of partitions. Each partition is an ordered, immutable sequence of messages that is continually appended to -- think of like a log file. Partitions are way Kafka scales as they allow for messages to be consumed in parallel.

A topic is just a logical grouping of partitions. **Topics are the way you publish and subscribe to data in Kafka.** When you publish a message, you publish it to a topic, and when you consume a message, you consume it from a topic. **Topics are always multi-producer;** that is, a topic can have zero, one, or many producers that write data to it.

So what is the difference between a topic and a partition?

A topic is a logical grouping of messages. A partition is a physical grouping of messages. A topic can have multiple partitions, and each partition can be on a different broker. Topics are just a way to organize your data, while partitions are a way to scale your data.

Last up we have our **producers** and **consumers**. Producers are the ones who write data to topics, and consumers are the ones who read data from topics. While Kafka exposes a simple API for both producers and consumers, the creation and processing of messages is on you, the developer. Kafka doesn't care what the data is, it just stores and serves it.

Importantly, you can use **Kafka as either a message queue or a stream**. Frankly, the distinction here is minor. **The only meaningful difference** is with how consumers interact with the data. **In a message queue, consumers read messages from the queue and then acknowledge that they have processed the message. In a stream, consumers read messages from the stream and then process them, but they don't acknowledge that they have processed the message. This allows for more complex processing of the data.**

How Kafka Works

When an event occurs, the producer formats a message, also referred to as a **record**, and sends it to a **Kafka topic**. A message consists of one required field, the value, and three optional fields: a key, a timestamp, and headers. **The key is used to determine which partition the message is sent to, and the timestamp is used to order messages within a partition.** Headers, like HTTP headers, are key-value pairs that can be used to store metadata about the message.

While not strictly required, the key is used to determine which partition the message is sent to. If you don't provide a key, Kafka will randomly assign the message to a partition. **So when designing a large, distributed system like you're likely to be asked about in your interview, you'll want to use keys to ensure that messages are processed in order, and the choice of that key is important. More on this later.**

As a quick example, here is how we might put a message on the topic `my-topic` using the Kafka command line tool

kafka-console-producer:

```
kafka-console-producer --broker-list localhost:9092 --topic my_topic --property "parse.key=true" --property "key.separator=:"
```

> key1: Hello, Kafka with key!

> key2: Another message with a different key

The `--property "parse.key=true"` and `--property "key.separator=:"` flags are used to specify that the key-value pairs are separated by a colon.

We can see what the same would look like using `kafkajs`, a popular Node.js client for Kafka:

```
// Initialize the Kafka client
const kafka = new Kafka({
  clientId: 'my-app',
  brokers: ['localhost:9092']
})

// Initialize the producer
const producer = kafka.producer()

const run = async () => {
  // Connecting the producer
  await producer.connect()

  // Sending messages to the topic 'my_topic' with keys
  await producer.send({
    topic: 'my_topic',
    messages: [
      { key: 'key1', value: 'Hello, Kafka with key!' },
      { key: 'key2', value: 'Another message with a different
key' }
    ],
  })
}
```

When a message is published to a Kafka topic, Kafka first determines the appropriate partition for the message. This partition selection is critical because it influences the distribution of data across the cluster. This is a two-step process:

1. **Partition Determination:** Kafka uses a partitioning algorithm that hashes the message key to assign the message to a specific partition. If the message does not have a key, Kafka can either round-robin the message to partitions or follow another partitioning logic defined in the producer configuration. This ensures that messages with the same key always go to the same partition, preserving order at the partition level.

2. **Broker Assignment:** Once the partition is determined, Kafka then identifies which broker holds that particular partition. The mapping of partitions to specific brokers is managed by the Kafka cluster metadata, which is maintained by the Kafka controller (a role within the broker cluster). The producer uses this metadata to send the message directly to the broker that hosts the target partition.

Each partition in Kafka functions essentially as an append-only log file. Messages are sequentially added to the end of this log, which is why Kafka is commonly described as a distributed commit log. This append-only design is central to Kafka's architecture, providing several important benefits:

1. **Immutability:** Once written, messages in a partition cannot be altered or deleted. This immutability is crucial for Kafka's performance and reliability. It simplifies replication, speeds up recovery processes, and avoids consistency issues common in systems where data can be changed.
2. **Efficiency:** By restricting operations to appending data at the end of the log, Kafka minimizes disk seek times, which are a major bottleneck in many storage systems.
3. **Scalability:** The simplicity of the append-only log mechanism facilitates horizontal scaling. More partitions can be added and distributed across a cluster of brokers to handle increasing loads, and each partition can be replicated across multiple brokers to enhance fault tolerance.

Each message in a Kafka partition is assigned a unique offset, which is a sequential identifier indicating the message's position in the partition. This offset is used by consumers to track their progress in reading messages from the topic. As consumers read messages, they maintain their current offset and periodically commit this offset back to Kafka. This way, they can resume reading from where they left off in case of failure or restart.

How Kafka Works

Once a message is published to the designated partition, Kafka ensures its durability and availability through a robust replication mechanism. Kafka employs a leader-follower model for replication, which works as follows:

1. **Leader Replica Assignment:** Each partition has a designated leader replica, which resides on a broker. This leader replica is responsible for handling all read and write requests for the partition. The assignment of the leader replica is managed centrally by the cluster controller, which ensures that

each partition's leader replica is effectively distributed across the cluster to balance the load.

2. **Follower Replication:** Alongside the leader replica, several follower replicas exist for each partition, residing on different brokers. These followers do not handle direct client requests; instead, they passively replicate the data from the leader replica. By replicating the messages received by the leader replica, these followers act as backups, ready to take over should the leader replica fail.
3. **Synchronization and Consistency:** Followers continuously sync with the leader replica to ensure they have the latest set of messages appended to the partition log. This synchronization is crucial for maintaining consistency across the cluster. If the leader replica fails, one of the follower replicas that has been fully synced can be quickly promoted to be the new leader, minimizing downtime and data loss.
4. **Controller's Role in Replication:** The controller within the Kafka cluster manages this replication process. It monitors the health of all brokers and manages the leadership and replication dynamics. When a broker fails, the controller reassigns the leader role to one of the in-sync follower replicas to ensure continued availability of the partition.

Last up, consumers read messages from Kafka topics using a **pull-based model**.

Unlike some messaging systems that push data to consumers, Kafka consumers actively poll the broker for new messages at intervals they control. As explained by [Apache Kafka's official documentation](#), this pull approach was a deliberate design choice that provides several advantages: it lets consumers control their consumption rate, simplifies failure handling, prevents overwhelming slow consumers, and enables efficient batching.

To round out our earlier example, here is how we might consume messages from the `my-topic` topic using the Kafka command line tool `kafka-console-consumer`:

```
kafka-console-consumer --bootstrap-server localhost:9092 --topic my_topic --from-beginning
```

Output

```
key1: Hello, Kafka with key!
```

```
key2: Another message with a different key
```

Similarly, with `kafka.js`, we can consume messages from the `my_topic` topic:

```
// Initialize the Kafka client
```

```
const kafka = new Kafka({  
  clientId: 'my-app',  
  brokers: ['localhost:9092']  
})
```

```

}))

// Initialize the consumer
const consumer = kafka.consumer({ groupId: 'my-group' })

const run = async () => {
  // Connecting the consumer
  await consumer.connect()

  // Subscribing to the topic 'my_topic'
  await consumer.subscribe({ topic: 'my_topic' })

  // Consuming messages
  await consumer.run({
    eachMessage: async ({ topic, partition, message }) => {
      console.log({
        value: message.value.toString(),
        key: message.key.toString()
      })
    },
  })
}

```

Tying it all together, we get something like this:

Output

When to use Kafka in your interview

Kafka can be used as either a message queue or a stream.

The **key difference** between the two lies in how consumers interact with the data. In a message queue, consumers typically pull messages from the queue when they are ready to process them. In a stream, consumers continuously consume and process messages as they arrive in real-time, similar to drinking from a flowing river.

Consider adding a message queue to your system when:

- You have processing that can be done asynchronously. YouTube is a good example of this. When users upload a video we can make the standard definition video available immediately and then put the video (via link) a Kafka topic to be transcoded when the system has time.

- You need to ensure that messages are processed in order. We could use Kafka for our virtual waiting queue in [Design Ticketmaster](#) which is meant to ensure that users are let into the booking page in the order they arrived.
- You want to decouple the producer and consumer so that they can scale independently. Usually this means that the producer is producing messages faster than the consumer can consume them. This is a common pattern in microservices where you want to ensure that one service can't take down another.

Streams are useful when:

- You require continuous and immediate processing of incoming data, treating it as a real-time flow. See [Design an Ad Click Aggregator](#) for an example where we aggregate click data in real-time.
- Messages need to be processed by multiple consumers simultaneously. In [Design FB Live Comments](#) we can use Kafka as a pub/sub system to send comments to multiple consumers.

What you should know about Kafka for System Design Interviews

There is a lot to know about Kafka. But we'll focus in on this bits that are most likely to be relevant to your system design interview.

This deep dive is rather exhaustive, especially as it pertains to the knowledge needed for an interview. Don't feel overwhelmed. If you're a junior or mid-level engineer, you likely won't need to know anything below this point. If you're a senior engineer, you should be familiar with some of the topics we're about to cover. Staff engineers and above would do well to know the majority of the topics below, but by no means is this knowledge required to pass an interview.

Scalability

Let's start by understanding the constraints of a single Kafka broker. It's important in your interview to estimate the throughput and number of messages you'll be storing in order to determine whether we need to worry about scaling in the first place.

First, there is no hard limit on the size of a Kafka message as this can be configured via `message.max.bytes`. However, it is recommended to keep messages under **1MB** to ensure optimal performance via reduced memory pressure and better network utilization.

It's a common **anti-pattern** in system design interviews to store large blobs of data in Kafka. Kafka is not a database, and it's not meant to store large files. It's meant to store small messages that can be processed quickly.

For example, when designing YouTube, we need to perform post-processing on videos after uploading to chunk and transcode them. **Naively, you might place the videos in Kafka so that the chunk/transcoding worker can pull them off the queue asynchronously and process them. This is not a good idea. Instead, you should store the videos in a distributed file system like S3 and place a message in Kafka with the location of the video in S3. This way, the Kafka message is small and serves as a pointer to the full video in S3.**

On good hardware, a single broker can store around 1TB of data and handle as many as 1M messages per second (this is very hand wavy as it depends on message size and hardware specs, but is a useful estimate). If your design does not require more than this, then scaling is likely not a relevant conversation.

In the case that you do need to scale, you have a couple strategies at your disposal:

1. **Horizontal Scaling With More Brokers:** The simplest way to scale Kafka is by adding more brokers to the cluster. This helps distribute the load and offers greater fault tolerance. Each broker can handle a portion of the traffic, increasing the overall capacity of the system. It's really important that when adding brokers you ensure that your topics have sufficient partitions to take advantage of the additional brokers. More partitions allow more parallelism and better load distribution. If you are under partitioned, you won't be able to take advantage of these newly added brokers.
2. **Partitioning Strategy:** This should be the main focus of your scaling strategy in an interview and is the main decision you make when dealing with Kafka clusters (since much of the scaling happens dynamically in managed services nowadays). You need to decide how to partition your data across the brokers. This is done by choosing a key for your messages. The partition is determined by hashing the key using a hash function (murmur2 by default) and taking the modulo with the number of partitions: $\text{partition} = \text{hash}(\text{key}) \% \text{num_partitions}$. If you choose a bad key, you can end up with hot partitions that are overwhelmed with traffic. Good keys are ones that are evenly distributed across the partition space.

It's worth noting that outside of an interview, many scaling consideration are made easy via managed Kafka services like Confluent Cloud or AWS MSK. These services handle much of the scaling for you, but you should still understand the underlying concepts.

When working with Kafka, you're usually thinking about scaling topics rather than the entire cluster. This is because different topics can have different requirements. For example, you may have a topic that is very high throughput and needs to be partitioned across many brokers, while another topic is low throughput and can be handled by a single broker. To scale a topic, you can increase the number of partitions, which will allow you to take advantage of more brokers.

How can we handle hot partitions?

Interviewers love to ask this question. Consider an [Ad Click Aggregator](#) where Kafka stores a stream of click events from when users click on ads. Naturally, you would start by partitioning by ad id. But when Nike launches their new LeBron James ad, you better believe that partition is going to be overwhelmed with traffic and you'll have a hot partition on your hands.

There are a few strategies to handle hot partitions:

1. **Random partitioning with no key:** If you don't provide a key, Kafka will randomly assign a partition to the message, guaranteeing even distribution. The downside is that you lose the ability to guarantee order of messages. If this is not important to your design, then this is a good option.
2. **Random salting:** We can add a random number or timestamp to the ad ID when generating the partition key. This can help in distributing the load more evenly across multiple partitions, though it may complicate aggregation logic later on the consumer side. This is often referred to as "salting" the key.
3. **Use a compound key:** Instead of using just the ad ID, use a combination of ad ID and another attribute, such as geographical region or user ID segments, to form a compound key. This approach helps in distributing traffic more evenly and is particularly useful if you can identify attributes that vary independently of the ad ID.
4. **Back pressure:** Depending on your requirements, one easy solution is to just slow down the producer. If you're using a managed Kafka service, they may have built-in mechanisms to handle this. If you're running your own Kafka cluster, you can implement back pressure by having the producer check the lag on the partition and slow down if it's too high.

Fault Tolerance and Durability

If you chose Kafka, one reason may have been because of its strong durability guarantees. But how does Kafka ensure that your data is safe and that no messages are lost?

Kafka ensures data durability through its replication mechanism. Each partition is replicated across multiple brokers, with one broker acting as the leader and others as followers. When a producer sends a message, it is written to the leader and then replicated to the followers. This ensures that even if a broker fails, the data remains available. Producer acknowledgments (`acks` setting) play a crucial role here. Setting `acks=all` ensures that the message is acknowledged only when all replicas have received it, guaranteeing maximum durability.

Depending on how much durability you need, you can configure the replication factor of your topics. The replication factor is the number of replicas that are maintained for each partition. A replication factor of 3 is common, meaning that each partition has 2 replicas. So if one broker fails, the data is still available on the other two and we can promote a follower to be the new leader.

But what happens when a consumer goes down?

Kafka is usually thought of as always available. You'll often hear people say, "Kafka is always available, sometimes consistent." This means that a question like, "what happens if Kafka goes down?" is not very realistic, and you may even want to gently push back on the interviewer if they ask this.

What is far more relevant and likely is that a consumer goes down. When a consumer fails, Kafka's fault tolerance mechanisms help ensure continuity:

1. **Offset Management:** Remember that partitions are just append-only logs where each message is assigned a unique offset. Consumers commit their offsets to Kafka after they process a message. This is the consumers way of saying, "I've processed this message." When a consumer restarts, it reads its last committed offset from Kafka and resumes processing from there, ensuring no messages are missed or duplicated.
2. **Rebalancing:** When part of a consumer group, if one consumer goes down, Kafka will redistribute the partitions among the remaining consumers so that all partitions are still being processed.

The trade-off you may need to consider in an interview is when to commit offsets. In [Design a Web Crawler](#), for example, you want to be careful not to commit the offset until you're sure the raw HTML has been stored in your blob storage. The more work a consumer has to do, the more likely you are to have to redo work if the consumer fails. For this reason, keeping the work of the consumer as small as possible is a good strategy -- as was the case in Web Crawler where we broke the crawler into 2 phases: downloading the HTML and then parsing it.

Handling Retries and Errors

While Kafka itself handles most of the reliability (as we saw above), our system may fail getting messages into and out of Kafka. We need to handle these scenarios gracefully.

Producer Retries

First up, we may fail to get a message to Kafka in the first place. Errors can occur due to network issues, broker unavailability, or transient failures. To handle these scenarios gracefully, Kafka producers support automatic retries. Here's a sneak peek of how you can configure them:

```
const producer = kafka.producer({
  retry: {
    retries: 5, // Retry up to 5 times
    initialRetryTime: 100, // Wait 100ms between retries
  },
  idempotent: true,
});
```

You'll want to ensure that you enable idempotent producer mode to avoid duplicate messages when retries are enabled. This just ensures that messages are only sent once in the case we incorrectly think they weren't sent.

Consumer Retries

On the consumer side, we may fail to process a message for any number of reasons. Kafka does not actually support retries for consumers out of the box (but [AWS SQS](#) does!) so we need to implement our own retry logic. One common pattern is to set up a custom topic that we can move failed messages to and then have a separate consumer that processes these messages. This way, we can retry messages as many times as we want without affecting the main consumer. If a given message is retried too many times, we can move it to a dead letter queue (DLQ). DLQs are just a place to store failed messages so that we can investigate them later.

Consumer Retries

You'll see in our [Web Crawler](#) breakdown that we actually opt for SQS instead of Kafka so that we could take advantage of the built-in retry and dead letter queue functionality without having to implement it ourselves.

Performance Optimizations

Especially when using Kafka as an event stream, we need to be mindful of performance so that we can process messages as quickly as possible. This first thing we can do is batch messages in the producer before sending them to Kafka. Again, this is just a simple configuration.

```
const producer = kafka.producer({
  batch: {
    maxSize: 16384, // Maximum batch size in bytes
    maxTime: 100,   // Maximum time to wait before sending a
    batch
  }
});
```

Another common way to improve throughput is by compressing the messages on the producer. This can be done by setting the `compression` option in the producer configuration. Kafka supports several compression algorithms, including GZIP, Snappy, and LZ4. Essentially, we're just making the messages smaller so that they can be sent faster.

```
const producer = kafka.producer({
  compression: CompressionTypes.GZIP,
});
```

Arguably the biggest impact you can have to performance comes back to your choice of partition key. The goal is to maximize parallelism by ensuring that messages are evenly distributed across partitions. In your interview, discussing the partition strategy, as we go into above, should just about always be where you start.

Retention Policies

Kafka topics have a retention policy that determines how long messages are retained in the log. This is configured via the `retention.ms` and `retention.bytes` settings. The default retention policy is to keep messages for 7 days.

In your interview, you may be asked to design a system that needs to store messages for a longer period of time. In this case, you can configure the retention policy to keep messages for a longer duration. Just be mindful of the impact on storage costs and performance.

Summary

Apache Kafka is an open-source, distributed event streaming platform engineered for high performance, scalability, and durability. It uses producers to send messages to topics, and consumers to read them, with messages being stored in ordered, immutable partitions across multiple brokers (servers). It is highly suited for real-time data processing and asynchronous message queuing in system design.

When it comes to scale, make sure you start by discussing your partitioning strategy and how you'll handle hot partitions. And remember, Kafka is always available, sometimes consistent